翁恺课 · 第 6 主题

W6 数组与函数 · 练习

对齐翁恺《程序设计入门——C语言》第 6 讲 · 参考答案含「思路 + 常见错法」· 请先独立完成再看答案

填空 W6-1

① 写一个函数要凑齐三样：____（告诉编译器"有这函数"）、____（写函数体"怎么算"）、____（在 main 里"用起来"）。返回类型写 `void` 表示____。

填空 W6-2

② 普通变量作参数是____（传值/传址），函数内改形参____影响实参；数组作参数传的是____，函数内改数组____影响原数组。

填空 W6-3

③ 调用 `swap(x, y)` 后外面的 `x`、`y` 没变，是因为函数拿到的是____；想要函数真正交换两个数，本讲解决不了，需要学____（W8）传地址。

改错 W6-4

④ 下面程序想封装一个"两数相加"的函数，有 2 处问题（1 个编译警告 + 1 处"能编译但逻辑不对"）。先自己找，再对照答案。

```c
/* debug_wk6_401.c — 封装加法（找 2 处问题）*/
#include <stdio.h>

int add(int a, int b);   /* 返回 int */

int main(void)
{
    printf("%d\n", add(3, 4));
    return 0;
}

int add(int a, int b)
{
    int s = a + b;
    /* ? 少了什么 */
}
```

编程 W6-5

⑤ 封装一个函数 `int sum_array(int a[], int n)` 求数组元素之和，在 main 里定义 {10, 20, 30, 40, 50} 并调用输出。示例输出应为 `总和 = 150`。

编程 W6-6

⑥ 封装 `int max_array(int a[], int n)` 求数组最大值，main 里传 {3, 9, 1, 12, 5, 8}（n=6）。示例输出应为 `最大值 = 12`。

专业场景 W6-7

⑦ 把 W5 的成绩统计重写成**函数版**：封装三个函数 `average`（平均，返回 double）、`max_score`（最高）、`count_fail`（不及格），main 只管定义数组 + 调用 + 打印。输入 {78, 92, 55, 61, 48} 应输出 `平均 = 66.80`，`最高 = 92`，`不及格 = 2`。

概念 W6-8

⑧ 结合讲义图 W6-2：为什么 `int` 作参数时函数内改不动外面（传值），而数组作参数时函数内改了外面就变？"传首地址"到底传的是什么？如果不小心在函数里对 `int` 也想"改外面"，C 的正确解法是什么？

参考答案

① 声明（prototype）、定义（definition）、调用（call）。`void` 表示**不返回任何值**（函数只管做事，不吐结果）。

思路：声明告诉编译器"名字+参数类型+返回类型"，定义写算法，调用真正用。常见错法：只有定义没声明就调用 → C99 报 `error: implicit declaration of function`；或忘写返回类型。

② 普通变量是**传值**，函数内改形参**不**影响实参；数组作参数传的是**首地址**（第一个元素的位置），函数内改数组**会**影响原数组。

思路：传值=复制一份（改副本原件没事）；传址=共享同一块内存（改的就是原件）。常见错法：以为传值能改外面（swap 换不动）；或以为数组也是复制（其实不是，函数里改了外面也变）。

③ 副本（实参的一份拷贝）。要真正交换需要学**指针**（W8），用指针把地址传给函数，函数通过地址改实参。

思路：这是"传值 vs 传址"的分水岭。常见错法：在函数里写 `a=b; b=t;` 后以为外面变了——只有传址（数组/指针）才改得动外面。

④ 问题：

1. `add` 定义里**漏了** `return s;`——返回类型是 `int` 却"走到函数尾没吐出值"，返回垃圾，这是逻辑错误（编译给警告）。
2. 顺带：`int s = a + b;` 因为没被用（没 return 出去），gcc 给 `unused variable` 警告。

gcc 诊断（`-Wall -Wextra` 实测原文）：

```
wk6_bug_ret.c:14:9: warning: unused variable 's' [-Wunused-variable]
   14 |     int s = a + b;
      |         ^
wk6_bug_ret.c:16:1: warning: control reaches end of non-void function [-Wreturn-type]
   16 | }
      | ^
```

诊断口径：`[-Wreturn-type]` 是**警告**（不是语法错误），编译能过但返回值是垃圾。修正：定义末尾加 `return s;`。教训：**非 void 函数每条路径都要 return**。

⑤ 参考代码（wk6_p5.c，gcc 实跑 0 警告）：

```c
#include <stdio.h>

int sum_array(int a[], int n);

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    printf("总和 = %d\n", sum_array(a, 5));
    return 0;
}

int sum_array(int a[], int n)
{
    int i, s = 0;
    for (i = 0; i < n; i++) s += a[i];
    return s;
}
```

实测输出：

```
总和 = 150
```

思路：函数收 `数组 + 长度`，内部累加后 `return s`。常见错法：忘了 return；或调用时只传 `a` 不传 `5`（长度传不进，循环越界）。

⑥ 参考代码（wk6_p6.c，gcc 实跑 0 警告）：

```c
#include <stdio.h>

int max_array(int a[], int n);

int main(void)
```

```
{
    int a[6] = {3, 9, 1, 12, 5, 8};
    printf("最大值 = %d\n", max_array(a, 6));
    return 0;
}

int max_array(int a[], int n)
{
    int i, m = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] > m) m = a[i];
    }
    return m;
}
```

实测输出：

```
最大值 = 12
```

思路：打擂台搬进函数，`m = a[0]` 当擂主、从 i=1 开始比。常见错法：把 `m` 初始化为 0（全负分时错）；或循环从 i=0 开始（多比一次 a[0] 无所谓但不干净）。

⑦ 参考代码（wk6_p7.c，gcc 实跑 0 警告）：

```
#include <stdio.h>

double average(int a[], int n);
int max_score(int a[], int n);
int count_fail(int a[], int n);

int main(void)
{
    int score[5] = {78, 92, 55, 61, 48};
    printf("平均 = %.2f，最高 = %d，不及格 = %d\n",
           average(score, 5), max_score(score, 5), count_fail(score, 5));
    return 0;
}

double average(int a[], int n)
{
    int i, s = 0;
    for (i = 0; i < n; i++) s += a[i];
    return (double)s / n;
}

int max_score(int a[], int n)
{
    int i, m = a[0];
```

```
    for (i = 1; i < n; i++) {
        if (a[i] > m) m = a[i];
    }
    return m;
}

int count_fail(int a[], int n)
{
    int i, c = 0;
    for (i = 0; i < n; i++) {
        if (a[i] < 60) c++;
    }
    return c;
}
```

实测输出：

```
平均 = 66.80，最高 = 92，不及格 = 2
```

思路：三个函数各管一件事，main 变"使用说明书"。常见错法：average 忘了 `(double)s / n`（整数除法截断成 66）；或三个函数里重复写同一份循环（没有真正"封装"）。

⑧ `int` 作参数时函数拿到的是**值的一份拷贝**（传值），函数内改的是这份拷贝，不碰外面的 x、y；数组作参数时函数拿到的是**数组首地址**（传址/传引用语义），通过地址访问的是原数组那块内存，所以改了外面也变。"传首地址"传的是**数组第一个元素在内存里的位置**（一个门牌号）。若想用函数改外面的 `int`，C 的解法是**传它的地址（指针）**——在 W8 学，那时 `swap` 就能真正交换了。

思路：一句话记——**int 传值、数组传址**。这决定了"函数内能不能改外面"。常见错法：把数组也当"复制一份"（其实不是），或反过来以为 int 也能传址（本讲不行，要指针）。

本练习册为自写原创变式题（对齐翁恺《程序设计入门——C语言》第 6 讲），仅作课堂伴读；改错题与编程题参考代码均用 gcc -Wall -Wextra 实测（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。原视频/教材版权归原作者所有。